CS3230 : Design and Analysis of Algorithms (Spring 2015)

Homework Set #2

[SOLUTION SKETCH]

[Do NOT give to future students: Let them have a chance to learn.]

R1. Using the most appropriate method, prove that
(a) $(\ln n)^5 = O(n^{0.2})$ **(b)** $2^n = O(n!)$
(*Hint:* Find good theorems to use to simplify your work. (Theorems are *magic potions*.))

Solution Sketch:
Use limit theorem + L' Hopital rule (differentiate (take derivative) of top and bottom).

$$\lim_{n\to\infty}\left(\frac{(\ln n)^5}{n^{0.2}}\right) = \lim_{n\to\infty}\left(\frac{5(\ln n)^4/n}{0.2/n^{0.8}}\right) = \lim_{n\to\infty}\left(\frac{5(\ln n)^4}{0.2n^{0.2}}\right)$$

So, after one round, we drop power of $(\ln n)$ from 5 to 4. Do one more round…

$$\lim_{n\to\infty}\left(\frac{5(\ln n)^4}{0.2n^{0.2}}\right) = \lim_{n\to\infty}\left(\frac{20(\ln n)^3/n}{(0.2)^2/n^{0.8}}\right) = \lim_{n\to\infty}\left(\frac{20(\ln n)^3}{(0.2)^2 n^{0.2}}\right)$$

So, you just need to do few more rounds of this…

(b) This one seems hard, because there appear NO way to differentiate $(n!)$
Instead, we use Lemma 2 of T2-D1. Take ln (log-base-e) of both functions first.
Then can use Stirling's approximation, and then the rest is smooth sailing.

S1. Modified from Problem 3-3, pp 61-62 of [CLRS] [Sorting out order of growth rates]
Rank the following functions in *increasing order of growth*; that is, if function $f(n)$ is *immediately* before function $g(n)$ in your list, then it should be the case that $f(n)$ is $O(g(n))$.

$g_1(n) = n!$	$g_2(n) = 5n^2(\lg n)+14$	$g_3(n) = n^{\lg n}$	$g_4(n) = 4n^{0.5}$
$g_5(n) = 2^{2(\lg n)}$	$g_6(n) = 3(\lg n)^2$	$g_7(n) = 2^n$	$g_8(n) = n^2(\lg\lg n)$

To simplify notations, we write $f(n) << g(n)$ to mean $f(n) = o(g(n))$ and $f(n) \equiv g(n)$ to mean $f(n) = \Theta(g(n))$. For example, the four functions n^2, n, $(2013n^2 + n)$ and n^3 could be sorted in increasing order of growth as follows: ($n << n^2 \equiv (2013n^2 + n) << n^3$).

Do not turn in proofs for this problem, but you should do them anyway just for practice.

Solution Sketch:

$$g_6(n) << g_4(n) << g_5(n) << g_8(n) << g_2(n) << g_3(n) << g_7(n) << g_1(n)$$

$g_6(n) = 3(\lg n)^2 \quad = \Theta((\lg n)^2)$

$g_4(n) = 4n^{0.5} \quad = \Theta(n^{0.5})$

$g_5(n) = 2^{2(\lg n)} = 2^{\lg(n*n)} = n*n = \Theta(n^2)$
$g_8(n) = n^2(\lg\lg n) \quad = \Theta(n^2(\lg\lg n))$
$g_2(n) = 5n^2(\lg n)+14 \quad = \Theta(n^2(\lg n))$

$g_3(n) = n^{\lg n} \quad = \Theta(n^{\lg n})$
$\quad = (2^{\lg n})^{\lg n} = 2^{(\lg n)*(\lg n)} << 2^n$ (since $(\lg n)^2 << n$)

$g_7(n) = 2^n \quad = \Theta(2^n)$
$g_1(n) = n! \quad = \Theta(n!)$

S2. [Really Solving Recurrences]

(a) Solve for $T(n)$, in Θ-notation, given that $T(n) = 4T(n/2) + \Theta(n^2\lg^3 n)$ for $n>1$, $T(1)=1$, by using the Master Theorem, the version given in the Lecture Notes.
[*Hint:* Carefully review lecture slides on Master Theorem.]

Solution Sketch:

$f(n) = \Theta(n^2\lg^3 n)$, $a=4$, $b=2$. So $n^{\log_b a} = n^{\log_2 4} = n^2$.
Since $f(n) = \Theta(n^2\lg^3 n) = \Theta(n^{\log_b a}\lg^3 n)$. This is **Case 2**. ($k=3$).
Hence, $T(n)= \Theta(n^2\lg^4 n)$.

(b) Solve the recurrence $T(n) = 4T(n/2) + 5n$ for $n>1$, $T(1)=1$ by using the recursion tree expansion method or by telescoping method.

Solution Sketch:

By Telescoping: Here, we define $h = \lg n$, and assume $T(1) = 5$ (a constant)

$$\begin{aligned}
T(n) &= 5n + 4T(n/2) \\
&= 5n + 4\cdot [5(\tfrac{n}{2})] + 4^2\cdot T(\tfrac{n}{2^2}) \\
&= 5n + 2(5n) + 4^2\cdot [5(\tfrac{n}{2^2})] + 4^3\cdot T(\tfrac{n}{2^3}) \\
&= 5n[1+2] + 2^2[5n] + 4^3\cdot [5(\tfrac{n}{2^3})] + 4^4\cdot T(\tfrac{n}{2^4}) \\
&= 5n[1+2+2^2+2^3] + \ldots + 4^h\cdot T(1) \\
&= 5n\left[1+2+2^2+2^3+\ldots+2^h\right] \\
&= 5n\cdot (2n) = \Theta(n^2)
\end{aligned}$$

Note that $2^h = n$, and so $4^h\, T(1) = (2^h)(2^h)(5) = (2^h)5n$

By Recursion Tree: Here, we define $h = \lg n$, and assume $T(1) = 5$ (a constant)

First, draw the recursion tree: DIY

Work Done at Level 0 (root):	$(5n)$
Work Done at Level 1:	$2(5n) = 4*(5(n/2))$
Work Done at Level 2:	$2^2(5n) = 4^2*(5(n/4))$
Work Done at Level 3:	$2^3(5n) = 4^3*(5(n/8))$
. . .	. . . . . .
Work Done at leaf Level:	$2^h(5n) = 4^h*T(1) = 4^h*(5) = 2^h(5n)$

Total done at all levels: $= 5n\left[1+2+2^2+2^3+\ldots+2^h\right] = 5n(2n) = \Theta(n^2)$

(c) Solve for $U(n)$, given the recurrence $U(n) = \left[\frac{2}{(n-1)}\sum_{k=1}^{n-1} U(k)\right] + 5n$.

Solution Sketch:

This is NOT hard. Just a bit tedious if you have NOT done it before.
Just "*reuse the method*" that was used to analyze QuickSort to get 1.386($n$lg n) avg-case as given in the lecture notes. (Yes, read the lecture notes first.)
Remember the two key Big-steps: 1. Get rid of full history, 2. Get it to "telescope".

Note: If you are still not good-friend with summation notation, write the sum out in full.

First multiply across by (n–1), make it less-ugly.

$$(n-1)U(n) = \left[2\sum_{k=1}^{n-1} U(k)\right] + 5n(n-1)$$

Now, Big-Step 1: Get rid of full history

$$(n-1)U(n) = 2\left[U(1)+U(2)+U(3)+...+U(n-2)+U(n-1)\right] + 5n(n-1)$$
$$(n-2)U(n-1) = 2\left[U(1)+U(2)+U(3)+...+U(n-2)\right] + 5(n-1)(n-2)$$

Subtract bottom from top, and just do the math

$$(n-1)U(n) = nU(n-1) + 10(n-1)$$

Now, Big-Step 2: Get it to "telescope". Divide by n(n–1)

$$\frac{U(n)}{n} = \frac{U(n-1)}{(n-1)} + \frac{10}{n}$$

Now, do the math of the telescoping to get

$$U(n) = 10nH(n) + O(1) = 13.86(n\lg n) + O(1) = \Theta(n\lg n)$$

S3. (Exercise 8.2-1 on p. 196 of [CLRS]) [Exercising Counting-Sort]
Using Figure 8.2 (or lecture note diagram) as a model, illustrate the operation of COUNTING-SORT on the array $A[1..12]$ = [3, 2, 1, 7, 1, 7, 2, 5, 1, 5, 7, 4]. You can assume that the numbers in the array A are integers ranging from 1 to 7. Show your working.

Solution Sketch:

$A[1..12]$] = [3, 2, 1, 7, 1, 7, 2, 5, 1, 5, 7, 4] → $C[1..7]$ = [3, 2, 1, 1, 2, 0, 3]
↓
$B[1..12]$] = [1, 1, 1, 2, 2, 3, 4, 5, 5, 7, 7, 7] ← $C'[1..7]$ = [3, 5, 6, 7, 9, 9, 12]

S4. [Finding *the* Missing Integer] (From a past exam/quiz)
You are given an array A[1..n] of size n, where $n = (2^k - 1)$. You are told that the *unsorted* array $A[1..n]$ contains all the integers from 0 to n (inclusive), *except one*, but you don't know the missing number. For k=3, an example is $A[1..7]$ = [3, 0, 2, 6, 1, 4, 7]. For this example, the *missing number* is 5.

(a) Give a $\Theta(n)$ algorithm to find the missing integer.
[*Hint:* Use an additional array $B[0..n]$ if necessary.]

SOLUTION: We use array $B[0..n]$ to count the items in $A[1..n]$ (use the first two loops of the Counting Sort). Then, from B, find the element with 0-count.

```
procedure FIND-MISSING; (* recursive procedure, v is a vertex in tree *)
begin
    for i ← 0 to n
            do B[i] ← 0
    for j ← 1 to n
            do B[A[ j]] ← B[A[ j]] + 1          ◁ B[i] = | {key = i} |
    for i ← 0 to n
            do if (B[i] = 0) then report i as missing number.  EXIT
end
```

Complexity analysis: Trivial $\Theta(n)$.

(b) Suppose that you are *now* told that you *cannot directly access* the integers in A with a single query operation. (*So, your algorithm from part* (*a*) *don't work now*.)

The elements in A are represented in binary and stored as k bits, namely,
$(b_{k-1}\ b_{k-2} \ldots b_1\ b_0)$ where each b_j is a binary bit. (*Note:* $k = \lg n$)

You can make the following query operation:

BIT-QUERY(j, i) : "fetch the jth bit of $A[i]$" (in constant $\Theta(1)$ time)

which returns a 0 or a 1 in constant $\Theta(1)$ time.

To illustrate this, we show an example using the array $A[1..7]$ given above and we give some sample queries. Remember, the array $A[1..n]$ is hidden from you (not accessible).

Hidden Array:	**Sample Queries**
$A[1]$ = 0 1 1	BIT-QUERY(0,2) = 0
$A[2]$ = 0 0 0	
$A[3]$ = 0 1 0	BIT-QUERY(1,3) = 1
$A[4]$ = 1 1 0	
$A[5]$ = 0 0 1	BIT-QUERY(2,6) = 1
$A[6]$ = 1 0 0	
$A[7]$ = 1 1 1	

Using this BIT-QUERY operation, and additional storage if needed, describe an algorithm for finding the *missing integer* with $\Theta(n)$ worst-case running time. Also illustrate how your algorithm works on the above example.

[*Note:* If you can use k queries to find *all the bit* of $A[i]$ (for all the i), then *this step alone* will take $\Theta(nk) = \Theta(n\lg n)$ time, which is TOO SLOW. We want $\Theta(n)$ worst-case.]

SOLUTION PROCESS:
For this part, I take a developmental approach (develop solution, bit-by-bit). Enjoy.

Trivial Solution suggested in the Note:

To get the value of $A[i]$, do k BIT-QUERY(j, i) (for $j = k–1, k–2, \ldots, 1, 0$), and use Horner's rule to compute value of $A[i]$. Do this for all $A[i]$.
Total of nk queries, so total time for this step is $\Theta(nk) = \Theta(n\lg n)$ time. [too slow]
Then can call the algorithm from (a) above, which takes only $\Theta(n)$ time.
BUT, Algorithm is slow -- $\Theta(n\lg n)$ time, which is not fast enough. (get *very low marks*)

NEXT KEY IDEAS:

The idea behind this is *binary search*, but we need to figure out – *HOW exactly to do this binary search.* What question do we ask in order to "divide the problem into two"? And then, decide which half of the problem to recursively solve?

We know each integer $A[i]$ is represented as a k-bit binary string.
Suppose the missing integer M, has binary encoding $M = (m_{k-1}m_{k-2} \ldots m_1m_0)$.

Step A: (Determine m_{k-1})
First, we do the following:

```
for i=1 to n do {Count[0] ← 0;  Count[1] ← 0 }          (*from counting sort *)
for i=1 to n do {
   b ← BIT-QUERY(k–1, i)          (* "fetch the (k–1)st bit of A[i])" for all i      *)
   Count[b] ← Count[b] + 1;       (* Count them… *)
}
```

Analysis: Clearly n queries. And also total time $\Theta(n)$.

Then at the end of this computation, if we look at Count[0] and Count[1].
What are their values? **STOP READING**, and think it one out *first*.

Yes, one of them is $2^{(k-1)}$, while the other is $2^{(k-1)}–1$. Now, why is that so?
[*Hint:* If $m_{k-1} = 0$, which one (of Count[0] or Count[1]) is $2^{(k-1)}$.
And if Count[1] $= 2^{(k-1)}$, then what is the value of m_{k-1}?]

In summary, using n queries, we determine value of m_{k-1}, (most significant bit)

Step B: (Determine all the rest.)
What do we do next?
Well, we can do the same thing with the $(k–2)^{th}$ bit. Can we not?
In that case, we can also spend n queries to determine the value of m_{k-2}.

And *then*, we can do the same for all the other bits of M.
This will take a total of $nk = n(\lg n)$ queries. And algorithm takes $\Theta(n\lg n)$ time.

Summary: With **Step A** and **Step B**, we solved the problem.
But, we used too many queries.

The question wants an algorithm that asks $\Theta(n)$ queries.

So, how can we do better?

To do better, we cannot spend n queries in each round of **Step B**.

How about making a small change to **Step A**?
Incorporate Divide-and-Conquer idea during **Step A**;
Suppose we borrow the *Partitioning* idea (Quicksort), and call this **Step A'**.

Step A': We "partition" the array $A[1..n]$ based on the bit #(k–1).

But, we cannot swap the items $A[i]$ and $A[j]$. Why?

So, we create two lists: (*important data structure*)
List[0] to store those indices j with $A[j]$=0,
List[1] to store those indices j with $A[j]$=1.
During partitioning, we merely insert (or append) j into List[0] or List[1] depending on value of bit-#(k–1)-of-$A[j]$ and also increment the size of each list. This takes $\Theta(1)$ per insert/append.

After "partitioning", we know Count[0], Count[1] and we know m_{k-1}. ☺

And we also know which "half" the missing number M belongs to.
Then we can recursively find m_{k-2} in *the correct half*, and so on.

Just a *small change* in the idea. But, we have reduced the problem size to $n/2$.

Let $T(n)$ = number of queries to solve this problem for size n.
Then we have, $T(n) = T(\underline{n}/2) + \Theta(n)$, which gives us $T(n) = \Theta(n)$.

Hidden Array:	**Simulation of the Final algorithm**		
$A[1]$ = 0 1 1	Round 1:	List[0] = {1,2,3,5}	$m_2 = 1$
$A[2]$ = 0 0 0		List[1] = {4,6,7}	Recur on List[1];
$A[3]$ = 0 1 0			
$A[4]$ = 1 1 0	Round 2:	List[0] = {6}	$m_1 = 0$
$A[5]$ = 0 0 1		List[1] = {4,7}	Recur on List[0]
$A[6]$ = 1 0 0			
$A[7]$ = 1 1 1	Round 3:	List[0] = {6}	$m_0 = 1$
		List[1] = {}	M = $(101)_2$ = 5

Now, don't you LOVE the problem?